

Deadline. The homework is due **11:59pm, Tue 04/27, 2026**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools.** If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here <https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/>

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

1 Programming: Parallel Prefix Sum

In this problem, we are going to use a parallel **exclusive scan** algorithm to compute **(exclusive) prefix sum**. Slightly different from the “(inclusive) prefix sum” problem in class, the exclusive prefix sum of an array A is another array B , such that $B[i] = \sum_{j=0}^{i-1} A[j]$, and $B[0] = 0$. You will find that the algorithmic idea can be very similar to the inclusive version. For example, you can first compute the inclusive version and subtract $A[i]$ from each $B[i]$. However, you can probably find more efficient ways to directly deal with the exclusive version.

We have learned two algorithms in class, and another one (with blocking) was introduced in your written homework 1. In written homework 1, we also introduced another version, which split the array into blocks to obtain parallelism. You will probably find the blocked version easier to implement. You can choose any one of them to implement scan.

You can find the template file of this assignment by using this invitation link:

<https://classroom.github.com/a/GwGcrIcM>

The instructions are available in the readme file.

Again, we will only use your header file (.h), and plug in our own test file. You can change anything in the repository (.cpp, makefile, etc.) to debug and test on your own, but any changes in these files will not be kept in our test. If you want to add helper functions, macros, etc., please put all of them in the header (.h) file. **For the same reason, please do not change the interface of the function (input and return value types).**

Some suggestions:

- For the best performance, you probably want to avoid allocating memory during the algorithm. That is to say, if you need extra space, allocate the memory in advance. This is because allocating memory in parallel can be expensive.
- This assignment can be useful in your future projects. For example, in Project 1, if you choose to implement a quicksort, an efficient scan algorithm is an important building block. Therefore, you can reuse your scan algorithm in future projects, and it is worth optimizing it in this assignment to get good performance.
- The deadline of this coding assignment is only one week before Project 1. Since it's likely that you'll use this code in your Project 1, I would suggest to make a runnable and reasonable version for scan much earlier than the deadline, so you can start working on Project 1 (or even Project 2) earlier. You can take the rest of the time optimizing them, but having a working version as early as possible is recommended.
- Don't forget to add granularity control and make sure your base case is not too small.

Your final grade for this part is determined by both the performance of your code and your report. If your algorithm finishes within 0.5s, you can get full marks for the running time part.

Running time: 7pts	
$\geq 0.8s$	$\leq 3pts$
0.7-0.8s	4pts
0.5-0.7s	5pts
$\leq 0.5s$	7pts
$\leq 0.4s$	up to 9pts + candies
Report: 3pts	

In your report, please specify your algorithm, some analysis, your optimizations, and experimental results to test/debug your algorithm.